# Informe Técnico de Auditoría — Checklist de Controles ISO/IEC 27002

**Producto 1 · Clase 3 — Evaluación de cumplimiento sobre la aplicación *Compra Segura* (NegocioSeguro)**

| | |
|---|---|
| **Asignatura** | Programación Orientada a Sitios Web |
| **Programa** | Tecnología en Desarrollo de Software para Negocios Digitales |
| **Institución** | Universidad de Manizales |
| **Docente** | Sergio Pinilla Valencia |
| **Equipo** | Pablo Nieto Pérez — *(agregar aquí el nombre completo de los demás integrantes)* |
| **Objeto auditado** | Aplicación web `compraseguras` — repositorio `https://github.com/pabloWIB/CompraSegura`, commit `b24a5de` |
| **Entorno de pruebas** | XAMPP / Apache + PHP + MariaDB (`http://localhost/compraseguras/`), phpMyAdmin |
| **Norma de referencia** | ISO/IEC 27002 (dominios A.9, A.10, A.12, A.14, A.16) |
| **Fecha del informe** | 9 de septiembre de 2026 |

---

## 1. Introducción y objetivo

Este informe da continuidad a la auditoría exploratoria de la Clase 2 (documento `HALLAZGOS.md` del repositorio), donde se identificaron y clasificaron nueve hallazgos de seguridad sobre la aplicación *Compra Segura*. En esta tercera actividad ya no se trata de **encontrar** fallas, sino de **medir el nivel de cumplimiento** de la aplicación frente a cinco dominios de control del estándar ISO/IEC 27002, aportando para cada uno evidencia concreta y verificable en el código fuente o en la base de datos.

Adicionalmente se documenta y justifica un hallazgo específico sobre el principio de **autenticidad** en el formulario de inicio de sesión, se responden las dos preguntas de la discusión final y se emite una respuesta formal a la objeción planteada en el reto del "Auditor Externo".

**Alcance:** los archivos `conexion.php`, `login.php`, `registro.php`, `pedido.php`, `buscar_producto.php`, el script `compraseguras.sql` y las tablas `usuarios`, `productos` y `pedidos` de la base de datos `compraseguras`.

**Fuera de alcance:** infraestructura del servidor Apache/MySQL, sistema operativo anfitrión, red y respaldos.

---

## 2. Metodología

La evaluación se realizó combinando tres técnicas, de modo que **todo estado declarado en el checklist es reproducible por un tercero**:

1. **Revisión estática de código (SAST manual).** Lectura línea por línea de los cinco archivos PHP, citando archivo y número de línea.
2. **Búsqueda dirigida de controles ausentes.** Se ejecutaron búsquedas sobre todo el proyecto para demostrar la *ausencia* de una función de seguridad, no solo su no observación. Comandos usados como evidencia:

```bash # Búsqueda de controles de validación / cifrado / saneamiento — 0 resultados grep -rniE "prepare|bind_param|real_escape|password_hash|password_verify|filter_var|intval|htmlspecialchars|is_numeric" --include=*.php .

# Búsqueda de mecanismos de registro de eventos (logs) — 0 resultados grep -rniE "error_log|file_put_contents|fopen|syslog|logger" --include=*.php .

# Búsqueda de roles o niveles de privilegio — solo aparece el usuario 'admin' como dato, nunca como rol grep -rniE "rol|perfil|nivel|permiso" --include=*.php --include=*.sql . ```

El resultado **vacío** de las dos primeras búsquedas es, en sí mismo, la evidencia técnica de incumplimiento de A.10 y A.12. 3. **Auditoría de base de datos en phpMyAdmin.** Inspección del DDL y del contenido de la tabla `usuarios` mediante:

```sql SHOW CREATE TABLE usuarios; SELECT id, usuario, clave, LENGTH(clave) AS longitud FROM usuarios; SHOW TABLES FROM compraseguras; -- verificar si existe alguna tabla de auditoría/log ```

### Escala de calificación aplicada

| Estado | Criterio de asignación |
|---|---|
| **Cumple** | El control existe, está implementado en el código o la base de datos y es efectivo frente a la amenaza que pretende mitigar. |
| **Cumple parcialmente** | Existe un elemento del control (documental o técnico), pero es incompleto, no es efectivo por sí solo, o no cubre todos los objetivos del dominio. |
| **No cumple** | El control no existe, o existe una implementación que resulta inoperante frente a la amenaza. |

---

## 3. Checklist de controles ISO/IEC 27002

### Resumen ejecutivo

| Dominio | Objetivo de control evaluado | Estado | Riesgo residual |
|---|---|---|---|
| **A.9** | Control de acceso (gestión de claves y privilegios) | 🔴 **No cumple** | Crítico |
| **A.10** | Criptografía (cifrado de información sensible) | 🔴 **No cumple** | Crítico |
| **A.12** | Seguridad de las operaciones (registro de eventos) | 🔴 **No cumple** | Alto |
| **A.14** | Adquisición, desarrollo y mantenimiento (validación de entradas) | 🔴 **No cumple** | Crítico |
| **A.16** | Gestión de incidentes (procedimientos de respuesta) | 🟡 **Cumple parcialmente** | Alto |

**Nivel global de cumplimiento: 10 %** (0 dominios *Cumple*, 1 dominio *Cumple parcialmente* de 5 evaluados).

---

### A.9 — Control de acceso

**Objetivo:** limitar el acceso a la información y a las instalaciones de procesamiento; garantizar el acceso de usuarios autorizados e impedir el acceso no autorizado.

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.9.1 | Existe un punto de entrada controlado a la aplicación | ¿La aplicación fuerza autenticación antes de exponer funcionalidad? | **No cumple** | El proyecto **no contiene** `index.php` ni `.htaccess`. Listado real del repositorio: `buscar_producto.php`, `compraseguras.sql`, `conexion.php`, `HALLAZGOS.md`, `login.php`, `pedido.php`, `registro.php`. Al abrir `http://localhost/compraseguras/` no hay redirección a `login.php`; se muestra el listado de directorio de Apache y cualquier archivo es invocable directamente por URL. |
| A.9.2 | Registro y baja de usuarios | ¿El alta de usuarios valida los datos y evita duplicados? | **No cumple** | `registro.php:4-7` inserta directamente `$_POST['usuario']` sin verificar existencia previa. `compraseguras.sql:4-8`: la columna `usuario VARCHAR(50)` **no tiene restricción** `UNIQUE` **ni** `NOT NULL`, por lo que pueden coexistir dos usuarios con el mismo nombre. |
| A.9.3 | Gestión de contraseñas: exigencia de complejidad y longitud mínima | *"El sistema valida que las contraseñas nuevas cumplan con un mínimo de caracteres y su complejidad en el archivo `registro.php`"* | **No cumple** | `registro.php` completo tiene 15 líneas. Entre la recepción del dato (`$clave = $_POST['clave'];`, línea 5) y su inserción (línea 6) **no existe ninguna instrucción de validación**: ni `strlen()`, ni `preg_match()`, ni `filter_var()`. El campo HTML (línea 13) tampoco declara `minlength`, `pattern` ni `required`. **Prueba ejecutada:** registro con usuario `test` y clave `1` → respuesta *"Usuario registrado con éxito."* y fila creada en la tabla. |
| A.9.4 | Gestión de privilegios / roles diferenciados | ¿Existen roles o niveles de acceso distintos? | **No cumple** | La tabla `usuarios` (`compraseguras.sql:4-8`) tiene únicamente tres columnas: `id`, `usuario`, `clave`. **No existe columna** `rol`, `perfil` **ni** `nivel`. La búsqueda `grep -rniE "rol\|perfil\|nivel\|permiso"` solo devuelve el literal `'admin'` como *dato* en `compraseguras.sql:11` y como contraseña de BD en `conexion.php:3`, nunca como control de autorización. Consecuencia: `admin` y `jperez` tienen exactamente los mismos privilegios en la aplicación. |
| A.9.5 | Restricción del acceso a la información (autorización por petición) | ¿Cada archivo verifica que exista sesión iniciada? | **No cumple** | `pedido.php:7` → `$usuario_id = $_SESSION['usuario_id'] ?? 0;`. El operador `??` sustituye la sesión ausente por `0` en lugar de bloquear la operación: un visitante **no autenticado** puede enviar un POST a `pedido.php` y el pedido se registra a nombre del usuario inexistente `0`. `buscar_producto.php` tampoco comprueba sesión. No existe ninguna instrucción `header('Location: login.php')` en todo el proyecto. |
| A.9.6 | Uso de utilitarios privilegiados / mínimo privilegio | ¿La aplicación se conecta a la BD con una cuenta de mínimo privilegio? | **No cumple** | `conexion.php:3` → `new mysqli("localhost", "root", "admin123", "compraseguras");`. La aplicación usa la cuenta `root` del gestor, con privilegios `ALL PRIVILEGES` sobre *todas* las bases de datos, y con una contraseña trivial escrita en claro en el código versionado en GitHub. |

> **Veredicto A.9 — NO CUMPLE.** No existe ninguna política de contraseñas, ni segregación de privilegios, ni verificación de sesión, ni principio de mínimo privilegio. Los seis puntos de control evaluados están ausentes.

---

## A.10 — Criptografía

**Objetivo:** asegurar el uso apropiado y eficaz de la criptografía para proteger la confidencialidad, autenticidad e integridad de la información.

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.10.1 | Protección criptográfica de credenciales almacenadas | ¿La columna `clave` de la tabla `usuarios` almacena hashes? | **No cumple** | **Auditoría en phpMyAdmin.** Consulta `SELECT id, usuario, clave, LENGTH(clave) FROM usuarios;` devuelve: `1 \| admin \| admin123 \| 8` `2 \| jperez \| miclave2024 \| 11` Las contraseñas son **legibles en texto plano**. Origen del dato: `compraseguras.sql:10-12`. |
| A.10.2 | Diseño de la estructura de datos acorde al algoritmo de cifrado | ¿El esquema soportaría un hash moderno? | **No cumple** | `SHOW CREATE TABLE usuarios;` → `clave VARCHAR(50)`. Un hash `bcrypt` generado por `password_hash()` ocupa **60 caracteres** y un SHA-256 en hexadecimal 64. El campo es *físicamente incapaz* de almacenar un hash estándar sin truncarlo, lo que demuestra que el cifrado nunca fue contemplado en el diseño. |
| A.10.3 | Uso de funciones criptográficas en el código | ¿Se invoca alguna función de hash o cifrado? | **No cumple** | `grep -rniE "password_hash\|password_verify\|hash(\|md5\|sha1\|openssl_\|crypt("` sobre todos los `.php` → **0 resultados**. La verificación de identidad en `login.php:8` se hace comparando la cadena en claro dentro del SQL: `... AND clave = '$clave'`. |
| A.10.4 | Protección de la información en tránsito | ¿El envío de credenciales viaja cifrado? | **No cumple** | El formulario de `login.php:21-25` publica por `POST` sobre `http://localhost/...` sin TLS y sin ninguna configuración de HTTPS ni cabecera `Strict-Transport-Security` en el proyecto. Las credenciales viajan en claro y son legibles con un analizador de tráfico. |
| A.10.5 | Gestión de claves y secretos | ¿Los secretos están fuera del código fuente? | **No cumple** | La contraseña del gestor de base de datos está escrita literalmente en `conexion.php:3` y **publicada en el repositorio Git**. No existe archivo `.env`, ni variables de entorno, ni `.gitignore`. |

> **Veredicto A.10 — NO CUMPLE.** No existe uso alguno de criptografía en la aplicación. Este dominio no tiene implementación parcial: su nivel es cero, tanto en reposo, como en tránsito, como en la gestión de secretos.

---

## A.12 — Seguridad de las operaciones

**Objetivo:** asegurar operaciones correctas y seguras; registrar eventos y generar evidencia (A.12.4 *Registro y seguimiento*).

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.12.1 | Registro de eventos (inicios de sesión exitosos) | ¿Algún archivo registra quién inició sesión? | **No cumple** | En `login.php:11-15`, tras una autenticación exitosa el sistema únicamente asigna variables de sesión e imprime `"Bienvenido, ..."`. **No escribe en ningún archivo ni tabla.** |
| A.12.2 | Registro de intentos fallidos de acceso | ¿Se registran los intentos fallidos? | **No cumple** | `login.php:16-18` — la rama `else` ejecuta un único `echo "Usuario o clave incorrectos.";`. No hay contador, ni marca de tiempo, ni bloqueo por reintentos, ni registro de la IP de origen. Esto permite ataques de fuerza bruta **indetectables**. |

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.12.3 | Existencia de un mecanismo de logging en el proyecto | ¿Se usa alguna función de escritura de log? | **No cumple** | `grep -rniE "error_log\|file_put_contents\|fopen\|syslog\|logger"` sobre todos los `.php` → **0 resultados**. No existe carpeta `/logs` ni archivo `.log` en el repositorio. |
| A.12.4 | Registro de actividades del administrador y de operaciones sensibles | ¿Se audita la creación de usuarios o de pedidos? | **No cumple** | `registro.php:6-8` inserta un usuario nuevo sin dejar traza de quién, cuándo ni desde dónde. La única marca temporal del sistema es `pedidos.fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP` ( `compraseguras.sql:32` ), que es un **dato de negocio de la transacción, no un registro de auditoría**: no identifica origen, no es inmutable y no cubre ningún evento de seguridad. |
| A.12.5 | Protección de la información de registro | ¿Los registros están protegidos contra alteración? | **No cumple** | No aplica por inexistencia del control anterior. Además, al operar la aplicación como `root` ( `conexion.php:3` ), cualquier inyección SQL exitosa podría borrar o alterar cualquier traza que llegara a existir. |
| A.12.6 | Gestión de errores / no divulgación de información técnica | ¿Los errores se registran en lugar de mostrarse al usuario? | **No cumple** | `conexion.php:4-6` → `die("Error de conexión: " . $conn->connect_error);` imprime el error del gestor **en el navegador del visitante** en lugar de registrarlo, revelando información de infraestructura. |

> **Veredicto A.12 — NO CUMPLE.** La aplicación es **ciega**: no puede responder a la pregunta "¿quién entró, cuándo y desde dónde?". La ausencia total de trazabilidad implica que un incidente de seguridad ocurrido hoy sería **forensemente irreconstruible**.

---

## A.14 — Adquisición, desarrollo y mantenimiento de sistemas

**Objetivo:** garantizar que la seguridad sea parte integral del ciclo de vida del desarrollo, en particular la validación de los datos de entrada y la seguridad de las transacciones de los servicios de aplicación.

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.14.1 | Validación de datos de entrada | ¿El código valida lo que el usuario envía antes de usarlo? | **No cumple** | `pedido.php:4-6` toma `producto_id` , `precio` y `cantidad` de `$_POST` **sin ninguna comprobación de tipo, rango o signo**. `buscar_producto.php:3` toma `$_GET['q']` con un simple `isset()` . `grep` de `filter_var\|intval\|is_numeric\|preg_match` → **0 resultados** en todo el proyecto. Una `cantidad` negativa ( `-5` ) genera un total negativo y se persiste sin error. |

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.14.2 | Uso de consultas parametrizadas (prevención de inyección) | ¿Las consultas separan código de datos? | **No cumple** | Las **cuatro** consultas del sistema concatenan entrada del usuario:<br>• `login.php:8` `SELECT * FROM usuarios WHERE usuario = '$usuario' AND clave = '$clave'`<br>• `registro.php:6` `INSERT INTO usuarios ... VALUES ('$usuario', '$clave')`<br>• `buscar_producto.php:5` `SELECT * FROM productos WHERE nombre LIKE '%$busqueda%'`<br>• `pedido.php:10` `INSERT INTO pedidos ... VALUES ('$usuario_id','$producto_id',...)`<br>`grep` de `prepare\|bind_param\|real_escape_string` → **0 resultados**. **Prueba ejecutada:** `buscar_producto.php?q='` produce error del gestor (fetch sobre `bool`), confirmando la inyectabilidad del parámetro. |
| A.14.3 | Seguridad de las transacciones de los servicios de aplicación | ¿El precio de la transacción se determina en el servidor? | **No cumple** | `pedido.php:17` → `<input type="hidden" name="precio" value="3000">` y `pedido.php:8` → `$total = $precio * $cantidad;`. **El precio es enviado por el cliente y el servidor lo acepta sin contrastarlo contra la tabla `productos`.** Prueba ejecutada modificando el campo oculto a `1`: el sistema responde *"Pedido registrado. Total a pagar: $1"* para el mismo producto de $3.000. Esto vulnera la **integridad** de la transacción comercial. |
| A.14.4 | Validación de datos de salida (codificación contextual) | ¿Se escapa lo que se imprime al navegador? | **No cumple** | `login.php:15` → `echo "Bienvenido, " . $fila['usuario'] . ...` y `buscar_producto.php:8` → `echo $fila['nombre'] ...`, ambos **sin `htmlspecialchars()`**. Como el registro tampoco filtra la entrada (A.14.1), un nombre de usuario con etiquetas HTML/JS se almacena y se ejecuta al mostrarse: **XSS almacenado**. |
| A.14.5 | Integridad de la petición / protección anti-CSRF | ¿Los formularios incluyen token de un solo uso? | **No cumple** | Ninguno de los cuatro formularios (`login.php:21`, `registro.php:11`, `pedido.php:15`, `buscar_producto.php:12`) incluye un campo token, ni existe generación o verificación de tokens en el código. |
| A.14.6 | Integridad referencial del modelo de datos | ¿La base de datos impone restricciones? | **No cumple** | `compraseguras.sql:26-33`: la tabla `pedidos` declara `usuario_id INT` y `producto_id INT` **sin claves foráneas (`FOREIGN KEY`) ni `NOT NULL`**. Se pueden insertar pedidos que referencian usuarios y productos inexistentes — de hecho el propio `pedido.php:7` lo hace con el valor `0`. La validación que debería estar en la capa de datos se delega íntegramente a un backend que tampoco la implementa. |

> **Veredicto A.14 — NO CUMPLE.** La seguridad no fue considerada en ninguna fase del desarrollo. El sistema confía en el 100 % de lo que le envía el cliente: la entrada, la salida, el precio y la integridad referencial.

---

## A.16 — Gestión de incidentes de seguridad de la información

**Objetivo:** asegurar un enfoque coherente y eficaz para la gestión de incidentes, incluida la comunicación de eventos y debilidades.

| # | Punto de control | Pregunta verificable | Estado | Evidencia técnica concreta |
|---|---|---|---|---|
| A.16.1 | Responsabilidades y procedimientos definidos | ¿Existe un procedimiento escrito de respuesta a incidentes? | **No cumple** | No existe en el repositorio ningún documento de procedimiento ( `SECURITY.md` , política, plan de respuesta, matriz de escalamiento). No hay roles asignados, ni tiempos de respuesta (SLA), ni criterio de escalamiento. |
| A.16.2 | Canal de reporte de eventos de seguridad | Si un cliente reportara hoy un problema, ¿por dónde lo haría? | **No cumple** | La aplicación no publica correo de contacto, formulario de reporte ni aviso legal. El único canal existente es el sistema de *Issues* de GitHub, que **no está documentado como canal de seguridad** y es público (un reporte allí divulgaría la vulnerabilidad antes de corregirla). |
| A.16.3 | Reporte de debilidades de seguridad | ¿Existe un mecanismo para registrar debilidades detectadas? | **Cumple parcialmente** | ✅ **Único control con evidencia positiva del informe.** El archivo `HALLAZGOS.md` (repositorio, commit `b24a5de` ) documenta 9 debilidades con archivo afectado, descripción, gravedad (Alta/Media/Baja) y remediación propuesta. Constituye un registro formal de debilidades. **Limitación:** es un producto académico puntual, no un proceso vivo: carece de responsable, de fecha, de estado de tratamiento y de seguimiento. |
| A.16.4 | Evaluación y clasificación de eventos | ¿Los eventos se clasifican por severidad? | **Cumple parcialmente** | `HALLAZGOS.md` incorpora una columna **"Gravedad"** con escala Alta/Media/Baja aplicada a los 9 hallazgos. Sin embargo, la escala no está definida formalmente (no se documentan los criterios de impacto y probabilidad que la sustentan) y solo se ha aplicado a hallazgos de auditoría, nunca a un incidente real. |
| A.16.5 | Respuesta a incidentes y recolección de evidencia | ¿Se puede investigar un incidente? | **No cumple** | **Dependencia directa con A.12:** al no existir ningún registro de eventos (0 resultados de `grep` sobre funciones de log), es **materialmente imposible** recolectar evidencia. Ante un reporte de acceso indebido, el equipo no podría determinar si ocurrió, cuándo, ni qué se accedió. |
| A.16.6 | Aprendizaje de los incidentes | ¿Los hallazgos alimentan mejoras? | **Cumple parcialmente** | `HALLAZGOS.md` cierra con una sección **"Nota"** que consolida el aprendizaje ("*casi todo se resume en dos cosas: usar consultas preparadas y hashear las contraseñas*"), priorizando las correcciones. Es un ejercicio real de lección aprendida, pero **no se ha traducido en ninguna corrección en el código**: el commit auditado sigue presentando los 9 hallazgos abiertos. |

> **Veredicto A.16 — CUMPLE PARCIALMENTE.** Se reconoce como evidencia positiva la existencia de un registro documentado y clasificado de debilidades ( `HALLAZGOS.md` ), que satisface de forma incipiente los objetivos de reporte, clasificación y aprendizaje. Sin embargo, **el ciclo de gestión de incidentes está roto en sus dos extremos**: no hay canal formal de entrada (A.16.2) ni capacidad de investigación en la salida (A.16.5). El resultado práctico es que la organización solo puede gestionar los incidentes que ella misma descubre leyendo su propio código, nunca los que sufre en producción.

---

## 4. Documentación y justificación del hallazgo de autenticidad

### 4.1 Situación observada en el inicio de sesión

**Ubicación:** `login.php` , líneas 4-18.

El formulario de acceso recibe dos campos ( `usuario` y `clave` ) y los inserta **sin ningún tratamiento** dentro de una cadena SQL:

```
$usuario = $_POST['usuario'];
$clave   = $_POST['clave'];

$sql = "SELECT * FROM usuarios WHERE usuario = '$usuario' AND clave = '$clave'";
$resultado = $conn->query($sql);

if ($resultado && $resultado->num_rows > 0) {
    $fila = $resultado->fetch_assoc();
    $_SESSION['usuario_id'] = $fila['id'];
    $_SESSION['usuario']    = $fila['usuario'];
    echo "Bienvenido, " . $fila['usuario'] . ". Acceso concedido.";
}
```

**Prueba ejecutada (bypass de autenticación):**

| Campo | Valor introducido |
|---|---|
| Usuario | `' OR ''='` |
| Clave | `' OR ''='` |

La consulta que llega finalmente al gestor es:

```
SELECT * FROM usuarios WHERE usuario = '' OR ''='' AND clave = '' OR ''=''
```

La condición `''=''` es siempre verdadera, por lo que la cláusula `WHERE` deja de filtrar y la consulta **devuelve todas las filas de la tabla** `usuarios`. Como el código solo comprueba `num_rows > 0` (línea 11) y luego toma la **primera fila** con `fetch_assoc()` (línea 12), el sistema:

1. Concede el acceso sin conocer ninguna contraseña.
2. Asigna a la sesión la identidad de la primera fila de la tabla, que según `compraseguras.sql:10-12` es `admin`.
3. Responde: *"Bienvenido, admin. Acceso concedido."*

**Resultado: cualquier persona en Internet puede iniciar sesión como `admin` sin credenciales.**

**Segunda situación observada (suplantación visual de identidad):** la columna `usuarios.usuario` ( `compraseguras.sql:6` ) **carece de restricción** `UNIQUE` y `registro.php` no verifica duplicados. Por lo tanto, un atacante puede registrar legítimamente una segunda cuenta llamada `admin` con la clave que él elija. Ante los ojos de cualquier otro usuario —y ante el propio mensaje de bienvenida de `login.php:15` — ambas cuentas son **indistinguibles**: el sistema muestra el mismo nombre para dos identidades diferentes.

### 4.2 Justificación: por qué esto compromete la autenticidad (y no la confidencialidad)

**Definición aplicada.** La **autenticidad** es la propiedad que garantiza que una entidad es *quien dice ser*, es decir, que la identidad reclamada por un sujeto ha sido genuinamente verificada. La **confidencialidad**, en cambio, garantiza que la información solo sea accesible a quien está autorizado a conocerla.

El hallazgo compromete la **autenticidad** por las siguientes razones técnicas:

1. **El sistema deja de verificar identidad y pasa a verificar existencia.** El control de `login.php:11` no pregunta *"¿es esta persona quien dice ser?"* sino *"¿la consulta devolvió al menos una fila?"*. Al poder manipularse la consulta misma, el atacante **no rompe el secreto de la contraseña: elimina la necesidad de la contraseña**. Nunca llega a conocer `admin123`; simplemente hace que la pregunta de verificación sea irrelevante. El mecanismo de prueba de identidad ha sido anulado, no descifrado.
2. **La identidad afirmada por el sistema es falsa.** Tras el bypass, `$_SESSION['usuario_id']` y `$_SESSION['usuario']` (líneas 13-14) contienen la identidad de `admin`, y **toda operación posterior del atacante quedará atribuida a `admin`**. El sistema, con toda su lógica funcionando "correctamente", está afirmando un hecho falso sobre quién está actuando. Esa afirmación falsa es precisamente la ruptura de la autenticidad, y arrastra consigo el **no repudio**: `admin` no podría negar de forma creíble unas acciones que el sistema registra a su nombre.
3. **La suplantación visual traslada el fallo al usuario final.** Con dos cuentas `admin` posibles, la identidad mostrada en pantalla deja de ser un identificador único. Un cliente que interactúe con el falso `admin` no tiene ningún elemento para distinguirlo del legítimo: el sistema le está presentando una identidad **no auténtica** como si lo fuera.

**Diferencia con la confidencialidad, sobre este mismo caso.** Si el atacante hubiera consultado la base de datos y leído `admin / admin123` en texto plano (hallazgo A.10), habría vulnerado la **confidencialidad**: obtuvo información secreta que no debía conocer. Es un problema distinto y separable. Aquí, en cambio, **el secreto sigue intacto y sin embargo el acceso se produce**: la falla no está en la protección del dato, sino en el proceso de verificación de la identidad. Dicho brevemente:

> **Confidencialidad rota = "supe tu contraseña". Autenticidad rota = "no necesité tu contraseña, y el sistema cree que soy tú".**

Ambas propiedades están afectadas en *Compra Segura*, pero por causas independientes: la confidencialidad por el almacenamiento en texto plano (A.10) y la autenticidad por la ausencia de una verificación de identidad confiable (A.9 + A.14). Corregir el almacenamiento de contraseñas **no repararía** el bypass, y corregir el bypass **no repararía** la exposición de las contraseñas: es la demostración práctica de que se trata de dos principios distintos.

### 4.3 Remediación propuesta para el hallazgo de autenticidad

```
// 1) Consulta preparada: los datos nunca se interpretan como código SQL
$stmt = $conn->prepare("SELECT id, usuario, clave FROM usuarios WHERE usuario = ?");
$stmt->bind_param("s", $usuario);
$stmt->execute();
$fila = $stmt->get_result()->fetch_assoc();

// 2) Verificación criptográfica real de la identidad (una sola fila, un solo usuario)
if ($fila && password_verify($clave, $fila['clave'])) {
    session_regenerate_id(true);                // 3) evita fijación de sesión
    $_SESSION['usuario_id'] = $fila['id'];
    echo "Bienvenido, " . htmlspecialchars($fila['usuario']);
} else {
    error_log("Login fallido: usuario={$usuario} ip={$_SERVER['REMOTE_ADDR']}"); // 4) traza (A.12)
    echo "Usuario o clave incorrectos.";        // 5) mensaje genérico, no revela cuál falló
}
```

Complementariamente, en base de datos: `ALTER TABLE usuarios MODIFY clave VARCHAR(255) NOT NULL, ADD UNIQUE KEY uk_usuario (usuario);`

---

## 5. Discusión final

### Pregunta 1 — Entre los dominios evaluados, ¿cuál debería corregirse primero y por qué?

**Respuesta del equipo: A.14 (validación de datos de entrada), y específicamente la adopción de consultas preparadas.**

Nuestra decisión se sustenta en tres criterios de priorización:

**a) Criterio de explotabilidad y alcance del daño.** El hallazgo de A.14 es explotable **de forma remota, anónima y sin ninguna herramienta especializada**: basta escribir `' OR ''='` en un campo de un formulario público. No requiere acceso previo, ni credenciales, ni acceso al servidor. En cambio, el hallazgo de A.10 (contraseñas en texto plano) exige que el atacante **ya haya conseguido acceso a la base de datos**; y los de A.12 y A.16 no son explotables por sí mismos: agravan un incidente, pero no lo causan. En términos de riesgo (probabilidad × impacto), A.14 tiene la probabilidad más alta y el impacto máximo.

**b) Criterio de causalidad: A.14 es la vulnerabilidad habilitante.** La inyección SQL no es un fallo aislado, es la **puerta que da acceso a todo lo demás**. A través de ella un atacante ejecuta `SELECT * FROM usuarios` y obtiene el contenido completo de la tabla; es decir, **el fallo de A.14 es el que convierte el fallo de A.10 en una fuga real de credenciales**. Agravado por A.9.6 (conexión como `root`), la inyección permite además leer o escribir en cualquier base de datos del servidor. Corregir A.10 sin corregir A.14 deja al atacante dentro del sistema; corregir A.14 le cierra la entrada principal.

**c) Criterio de costo-beneficio.** Sustituir las cuatro consultas concatenadas por sentencias preparadas es una corrección de bajo esfuerzo (aproximadamente veinte líneas de código, sin cambios en el modelo de datos ni migración) que **cierra simultáneamente los hallazgos 1, 2, 3 y 4 de** `HALLAZGOS.md` y neutraliza el bypass de autenticación. Ningún otro cambio del mismo tamaño reduce tanto riesgo.

**Orden de remediación propuesto:**

| Prioridad | Dominio | Acción | Justificación |
|---|---|---|---|
| 1 | **A.14** | Consultas preparadas + validación de tipos + precio leído del servidor | Cierra el vector de ataque remoto y no autenticado |
| 2 | **A.10** | `password_hash()` / `password_verify()` + `clave VARCHAR(255)` + migración | Limita el impacto de una fuga futura de la base de datos |
| 3 | **A.9** | Política de contraseñas, columna `rol`, verificación de sesión, usuario de BD con mínimo privilegio | Restablece el control de acceso y la segregación de privilegios |
| 4 | **A.12** | Registro de accesos exitosos y fallidos | Habilita la detección; sin esto no sabemos si ya nos atacaron |
| 5 | **A.16** | Canal de reporte, roles y procedimiento documentado | Formaliza la respuesta una vez existe capacidad de detección |

**Matiz importante:** aunque A.12 aparece en cuarto lugar por su prioridad de *corrección*, sostenemos que es el dominio de mayor prioridad **de instrumentación**. Sin registros, el equipo no puede afirmar que el sistema no haya sido ya comprometido, y no podrá verificar la eficacia de las correcciones 1 a 3. Recomendamos por tanto implementar el logging **en paralelo** con la corrección de A.14.

### Pregunta 2 — ¿Qué significa que un dominio quede en "Cumple parcialmente"? ¿Es aceptable?

En esta auditoría un solo dominio quedó en estado parcial: **A.16 (gestión de incidentes)**, y el análisis de ese caso concreto nos permite responder en tres niveles.

**a) Qué significa técnicamente.** "Cumple parcialmente" **no significa "medio seguro"**, ni es un punto intermedio en una escala continua de protección. Significa que *existe evidencia de que el control fue considerado*, pero que **su implementación no**

**alcanza a producir el efecto de seguridad que el control persigue**. En nuestro caso: el equipo produjo `HALLAZGOS.md`, un registro real de debilidades clasificadas por gravedad, lo cual satisface los objetivos de reporte y de aprendizaje (A.16.3, A.16.4, A.16.6). Pero la finalidad de A.16 no es documentar hallazgos: es **poder responder a un incidente**. Y esa capacidad no existe, porque falta el canal de entrada (A.16.2) y falta la evidencia para investigar (A.16.5, dependiente de A.12). El dominio tiene piezas, pero no tiene función.

**b) Por qué el estado parcial es más peligroso que el incumplimiento total.** Un "No cumple" es honesto: todo el mundo sabe que el control no está. Un "Cumple parcialmente" **genera una falsa sensación de cobertura**: al existir `HALLAZGOS.md`, la organización puede creer que "ya tiene gestionada la seguridad", cuando en realidad solo tiene un diagnóstico. Es precisamente el riesgo que observamos en el commit auditado: los nueve hallazgos están documentados con su solución escrita al lado, y **ninguno ha sido corregido en el código**. Documentar un riesgo sin tratarlo no reduce el riesgo; solo traslada la responsabilidad, porque a partir de ese momento la exposición es *conocida y aceptada*.

**c) ¿Es aceptable?** Es aceptable **como estado transitorio y explícitamente gestionado**, nunca como estado final. Para que un "Cumple parcialmente" sea defensible ante un auditor debe ir acompañado de tres elementos que hoy no existen en el proyecto: (i) un **plan de acción** con las tareas que cierran la brecha, (ii) un **responsable y una fecha** de cierre, y (iii) una **aceptación formal del riesgo residual** por parte de quien tiene autoridad para asumirlo. Sin esos tres elementos, "parcial" es funcionalmente equivalente a "no cumple", con el agravante de la falsa confianza.

**Conclusión del equipo:** convertimos nuestro propio hallazgo parcial en compromiso concreto — asignar `HALLAZGOS.md` a un responsable, añadir a cada hallazgo estado y fecha de cierre, publicar un `SECURITY.md` con el canal de reporte, y condicionar el cierre de A.16 a la implementación previa del registro de eventos de A.12.

---

## 6. Reto de profundización — Respuesta formal a la objeción del Auditor Externo

> **Objeción recibida:** *"Su informe está inflado. Todo lo que ustedes reportan proviene de una única causa: no usar consultas preparadas y no cifrar contraseñas. Están contando el mismo defecto cinco veces, en cinco dominios distintos, para simular un incumplimiento mayor del que realmente existe. Los dominios que evaluaron no son independientes."*

### 6.1 Reconocimiento parcial de la observación

El equipo auditor reconoce que existe **correlación causal** entre varios hallazgos: la inyección SQL (A.14) es efectivamente el vector que permite explotar el almacenamiento en texto plano (A.10), y la ausencia de registros (A.12) es la causa directa de la incapacidad de investigación (A.16.5). Esa correlación está documentada de forma explícita en este mismo informe (sección 5, pregunta 1) y es, de hecho, el fundamento de nuestro orden de remediación.

Sin embargo, **correlación causal no es equivalencia**. Sostenemos formalmente que la objeción confunde dos planos distintos del análisis de seguridad: el plano de la **causa técnica** y el plano del **objetivo de control**. ISO/IEC 27002 no está organizada por causas técnicas, sino por objetivos de control; y dos objetivos son independientes cuando **la corrección de uno no produce el cumplimiento del otro**. A continuación aplicamos esa prueba, dominio por dominio.

### 6.2 Prueba de independencia: el criterio de la remediación no transferible

> **Criterio formal aplicado:** *Si al corregir íntegramente el dominio X el dominio Y permanece incumplido, entonces X e Y son controles independientes y su evaluación separada es metodológicamente correcta.*

Aplicamos el criterio suponiendo la corrección **completa** de A.14 (todas las consultas parametrizadas y todas las entradas validadas), que es la causa única que el auditor externo propone:

| Dominio | ¿Queda resuelto al corregir A.14? | Demostración técnica |
|---|---|---|
| **A.10** Criptografía | ❌ **No** | Con consultas preparadas, la tabla `usuarios` **sigue almacenando** `admin123` **en texto plano** en una columna `VARCHAR(50)`. Cualquier respaldo extraviado, un acceso del administrador de base de datos, o una fuga por un canal distinto (phishing, servidor comprometido, `.sql` publicado en el repositorio — como ocurre hoy) expone todas las credenciales. La parametrización protege el canal de la aplicación; **no protege el dato en reposo**. Son controles con superficies de ataque distintas. |
| **A.12** Registro de eventos | ❌ **No** | Una aplicación con consultas preparadas **sigue sin escribir un solo registro**. Un atacante que realice 10.000 intentos de fuerza bruta contra `login.php` sería completamente indetectable, porque la rama `else` (línea 17) seguiría haciendo únicamente `echo`. Más aún: la parametrización *reduce* los vectores, pero no elimina el ataque por credenciales legítimas robadas — escenario que **solo** el registro de eventos puede detectar. |
| **A.9** Control de acceso | ❌ **No** | Con consultas preparadas, `registro.php` **seguiría aceptando la contraseña** `1`, la tabla `usuarios` **seguiría sin columna** `rol` (admin y cliente con idénticos privilegios), `pedido.php:7` **seguiría procesando pedidos sin sesión** mediante `?? 0`, y la aplicación **seguiría conectándose como** `root`. Ninguna de esas cuatro debilidades tiene relación alguna con la construcción de las consultas. |
| **A.16** Gestión de incidentes | ❌ **No** | El código más seguro del mundo no responde la pregunta organizacional: *¿a quién avisa un cliente que detecta un problema, quién lo atiende y en cuánto tiempo?* A.16 es un control **de proceso y de personas**, no de código. Ningún cambio en `login.php` puede crear un canal de reporte ni asignar un responsable. |

**Conclusión de la prueba:** ninguno de los cuatro dominios restantes se resuelve al corregir A.14. Por definición del criterio, los cinco dominios evaluados son **independientes**, y su calificación separada no constituye duplicación.

### 6.3 Argumentos adicionales de defensa

**1. La prueba inversa también se sostiene.** Si se corrigiera únicamente A.10 (hasheando todas las contraseñas con `bcrypt`), el bypass `' OR ''='` en `login.php` **seguiría funcionando exactamente igual**, porque la consulta se rompe *antes* de que la contraseña se compare. Este hecho —verificable en un minuto— demuestra empíricamente que A.10 y A.14 no son el mismo hallazgo: una vulnerabilidad que sobrevive a la corrección de otra es, necesariamente, una vulnerabilidad distinta.

**2. Los dominios miden propiedades de seguridad diferentes.** A.14 protege la **integridad** del procesamiento (que el sistema haga lo que debe con los datos que recibe); A.10 protege la **confidencialidad** del dato almacenado; A.12 protege la **trazabilidad** y sostiene el **no repudio**; A.9 protege la **autenticidad y la autorización**; A.16 protege la **capacidad de recuperación**. La sección 4.2 de este informe demuestra con un caso concreto que confidencialidad y autenticidad se rompen aquí por causas separables. Colapsar cinco propiedades en un solo hallazgo haría el informe **inauditable**: impediría asignar responsables distintos, estimar esfuerzos distintos y verificar cierres de forma independiente.

**3. La estructura de la norma es normativa, no opcional.** ISO/IEC 27002 define objetivos de control precisamente para que la evaluación sea **comparable entre organizaciones y auditable en el tiempo**. Un auditor no puede reagrupar dominios según su hipótesis sobre la causa raíz; debe evaluar cada objetivo de control frente a la evidencia. Reportar "un solo hallazgo de causa raíz" sería, de hecho, **la mala práctica**: ocultaría que cuatro objetivos de control de la norma están incumplidos por razones que subsisten después de tratar esa causa.

**4. Consecuencia práctica de aceptar la objeción.** Si el equipo aceptara la tesis del auditor externo y reportara un único hallazgo, la organización aplicaría una única corrección (consultas preparadas), **daría el caso por cerrado**, y quedaría operando con: contraseñas en texto plano, sin ninguna política de contraseñas, sin roles, con la aplicación conectada como `root`, sin ningún registro de auditoría y sin procedimiento de respuesta ante incidentes. El riesgo real permanecería casi intacto mientras la percepción de riesgo caería a cero. **Esa es exactamente la situación que una auditoría por dominios está diseñada para evitar.**

### 6.4 Declaración formal del equipo auditor

> El equipo auditor sostiene la calificación emitida sobre los cinco dominios. Reconoce y documenta la relación causal entre los hallazgos —relación que se refleja en el orden de remediación propuesto—, pero rechaza técnicamente la afirmación de que dicha relación convierta los hallazgos en uno solo. Aplicado el criterio de remediación no transferible, se demuestra que la corrección íntegra del dominio A.14, señalado por el auditor externo como causa única, **deja incumplidos los dominios A.9, A.10, A.12 y A.16**. Por consiguiente, los dominios evaluados son independientes, la evaluación separada es metodológicamente correcta, y el informe no presenta duplicación de hallazgos. El equipo queda a disposición para reproducir ante el auditor externo cualquiera de las pruebas descritas en la sección 2.

---

## 7. Conclusiones

1. La aplicación *Compra Segura* presenta un nivel de cumplimiento del **10 %** frente a los cinco dominios ISO/IEC 27002 evaluados: cuatro dominios en **No cumple** y uno en **Cumple parcialmente**.
2. La falla más grave es de **autenticidad**: el formulario de acceso puede ser evadido por completo con la cadena `' OR ''='`, otorgando privilegios de `admin` a cualquier visitante anónimo. Se trata de una vulneración de la identidad, no del secreto, y no se corrige cifrando contraseñas.
3. La corrección prioritaria es la **parametrización de las cuatro consultas SQL** (A.14), que con un esfuerzo mínimo cierra cinco de los nueve hallazgos previos y neutraliza el bypass de autenticación.
4. La ausencia total de registro de eventos (A.12) implica que **el equipo no puede afirmar hoy que el sistema no haya sido ya comprometido**, y limita gravemente cualquier respuesta a incidentes.
5. El único control con evidencia positiva ( `HALLAZGOS.md` ) demuestra capacidad de diagnóstico del equipo, pero **ningún hallazgo ha sido corregido en el código**: documentar un riesgo no lo reduce.

---

## 8. Anexo — Trazabilidad hallazgos Clase 2 → dominios ISO 27002 Clase 3

| # Hallazgo ( `HALLAZGOS.md` ) | Descripción | Gravedad | Dominio(s) ISO afectado(s) |
|---|---|---|---|
| 1 | Inyección SQL en el login | Alta | A.14.2, A.9.3 *(autenticidad)* |
| 2 | Inyección SQL en el registro | Alta | A.14.2 |
| 3 | Inyección SQL en la búsqueda | Alta | A.14.2 |
| 4 | Inyección SQL + precio enviado por el cliente | Alta | A.14.2, A.14.3 |
| 5 | Claves en texto plano | Alta | A.10.1, A.10.2, A.10.3 |
| 6 | Sin escape de salida (XSS) | Media | A.14.4 |
| 7 | Uso de `root` para la aplicación | Media | A.9.6 |
| 8 | `pedido.php` no valida sesión | Media | A.9.5 |
| 9 | Sin CSRF ni validación de datos | Baja | A.14.1, A.14.5 |

| # Hallazgo (`HALLAZGOS.md`) | Descripción | Gravedad | Dominio(s) ISO afectado(s) |
|---|---|---|---|
| — | *Nuevos en Clase 3:* ausencia total de logs | Alta | A.12 (completo) |
| — | *Nuevos en Clase 3:* sin procedimiento de incidentes | Alta | A.16.1, A.16.2, A.16.5 |
| — | *Nuevos en Clase 3:* sin `UNIQUE` en `usuarios.usuario` (suplantación) | Media | A.9.2, A.14.6 *(autenticidad)* |
| — | *Nuevos en Clase 3:* sin claves foráneas en `pedidos` | Media | A.14.6 |
| — | *Nuevos en Clase 3:* credenciales de BD versionadas en Git | Alta | A.10.5, A.9.6 |